

Practica 4: Clasificación de Texto

10 de julio de 2023

Dataset: Dataset de Encabezados de Noticias para la Detección de Sarcasmo.
Disponible en:
<https://www.kaggle.com/datasets/rmisra/news-headlines-dataset-for-sarcasm-detection>

1. Objetivo de la práctica.

Crear un modelo que pueda hacer análisis de texto para reconocer textos sarcásticos y no sarcásticos y prevenir la distribución de noticias falsas.

2. Conceptos.

Ya se habló de la librería Scikit-Learn en la Practica 3, así que toca hablar de las demás librerías que se usarán.

Natural Language Toolkit es una librería de procesamiento de lenguaje natural (NLP) la cuál usaremos para poder trabajar con los encabezados, limpiar los textos y poder usarlos en contexto.

El dataset consiste en una carpeta: Sarcasm_Headlines.Dataset, que a su vez cuentan con 2 archivos; Sarcasm_Headlines.Dataset.json y Sarcasm_Headlines.Dataset.v2.json.

3. Herramientas a usar.

- Computadora con acceso a internet.
- Los siguientes programas y librerías:
 - Python versión 3.8.8
 - TensorFlow versión 2.6.0
 - Anaconda versión 4.10.1
 - Jupyter-lab 3.014

- NumPy versión 1.21.4
- Pandas versión 1.2.4
- Matplotlib versión 3.3.4
- Zipfile versión 3.2.0
- Scikit-learn 1.3.0
- Natural Language Toolkit 3.7

4. Desarrollo.

4.1. Entender el problema.

Los estudios anteriores sobre detección de sarcasmo utilizan principalmente conjuntos de datos de Twitter recopilados mediante la supervisión basada en hashtags, pero estos conjuntos de datos son ruidosos en términos de etiquetas y lenguaje. Además, muchos tuits son respuestas a otros tuits y detectar sarcasmo en ellos requiere la disponibilidad de tuits contextuales.

Para superar las limitaciones relacionadas con el ruido en los conjuntos de datos de Twitter, este conjunto de datos de titulares de noticias para la detección de sarcasmo se recopila de dos sitios web de noticias;

- The Onion, una pagina de noticias que produce noticias de manera sarcástica (lo que sería ElDeforma en México).
- HuffPost, sitio de noticias reales con títulos de verdad

Este conjunto es superior al conjunto de datos de Twitter ya que los titulares de noticias están escritos de manera profesional y muy formal; incluso los de The Onion, para mantener su naturaleza de "parodia". Significa que la presencia de errores ortográficos es nula ni hay uso informal del idioma.

Otra ventaja es que los titulares contienen todo el contexto necesario para entenderlos, a diferencia de los tuits que son respuestas a tuits y no cuentan con el contexto necesario para poder entenderlos.

4.2. Criterio de evaluación.

Usaremos la exactitud (accuracy)..

4.3. Preparar los datos.

Debemos importar todas las librerías con las que estaremos trabajando durante la práctica.

```
1 import warnings
2 warnings.filterwarnings('ignore')
3
4 import re
```

```

5 import os
6 import string
7 import zipfile
8 import numpy as np
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 from matplotlib.ticker import PercentFormatter
12 from sklearn.model_selection import train_test_split

```

Estaremos trabajando con Tensorflow, por lo que es necesario comprobar que se esté usando nuestra GPU.

```

1 tf.config.list_physical_devices('GPU')
2
3
4 [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```

nltk (Natural Language Toolkit) es una librería de procesamiento de lenguaje natural (NLP) de Python. Ofrece herramientas para poder trabajar con NLP:

- Stopwords: Proporciona listas de palabras comunes en varios idiomas (como "el", "la", "de" en español) que se suelen eliminar en tareas de procesamiento de texto para reducir el ruido.
- Tokenización (punkt): Dividir un texto en palabras, oraciones u otras unidades significativas.
- Stemming y lematización (wordnet y own-1.4): Reducir palabras a sus raíces o formas base, lo cual es útil para entender mejor el significado de un texto. WordNet es una base de datos léxica en inglés y su complemento, Open Multilingual WordNet, incluye traducciones y otros recursos multilingüales.

Aquí vamos a descargar los paquetes para trabajar con ellos

```

1 import nltk
2 from nltk.corpus import stopwords
3 nltk.download("stopwords")
4 nltk.download('punkt')
5 nltk.download('wordnet')
6 nltk.download('omw-1.4')
7
8 [nltk_data] Downloading package stopwords to
9 [nltk_data] C:\Users\brugi\AppData\Roaming\nltk_data...
10 [nltk_data] Package stopwords is already up-to-date!
11 [nltk_data] Downloading package punkt to
12 [nltk_data] C:\Users\brugi\AppData\Roaming\nltk_data...
13 [nltk_data] Package punkt is already up-to-date!
14 [nltk_data] Downloading package wordnet to
15 [nltk_data] C:\Users\brugi\AppData\Roaming\nltk_data...
16 [nltk_data] Package wordnet is already up-to-date!
17 [nltk_data] Downloading package omw-1.4 to
18 [nltk_data] C:\Users\brugi\AppData\Roaming\nltk_data...
19 [nltk_data] Package omw-1.4 is already up-to-date!

```

Aquí definimos la ruta base de nuestros datos. Además, extraeremos las carpetas de los archivos y comprobaremos que se encuentren ahí.

```
1 Path = "../Practica 4/" #Depende de la ruta en tu maquina
2
3 os.makedirs(Path+"Sarcasm_Headlines_Dataset", exist_ok=True)
4 with zipfile.ZipFile('archive.zip', 'r') as archive:
5     # Extrae todo el contenido del archivo ZIP en el directorio
6     # actual
7     archive.extractall("Sarcasm_Headlines_Dataset")
8
9 os.listdir(Path)
10
11 ['.ipynb_checkpoints',
12 'archive.zip',
13 'Sarcasm_Headlines_Dataset',
14 'TF-Sarcasmo-Vacio.ipynb',
15 'TF-Sarcasmo.ipynb']
16
17 os.listdir(Path + "Sarcasm_Headlines_Dataset")
18
19 ['.ipynb_checkpoints',
20 'Sarcasm_Headlines_Dataset.json',
21 'Sarcasm_Headlines_Dataset_v2.json']
```

A diferencia de los ejercicios anteriores donde trabajábamos con imágenes ahora trabajaremos con texto. Lo prudente es revisar cuál es el contenido de los registros.

Cada registro en el archivo .json tiene 3 atributos:

- is_sarcastic : 1 si el registro es sarcástico, 0 si el registro es no sarcástico
- El título del artículo de noticias
- Enlace al artículo de noticias original.

```
1 df = pd.read_json(Path + "Sarcasm_Headlines_Dataset/"
2     "Sarcasm_Headlines_Dataset.json", lines=True)
3 df = df[['headline', 'is_sarcastic', 'article_link']]
4 df.head()
```

Una buena práctica es comprobar que no tengamos registros vacíos.

```
1 df["is_sarcastic"].isnull().any() # no missing values in
2     is_sarcastic column
3 df.headline.isnull().any() # no missing values in headline column
4 False
```

Podemos seguir revisando los datos para poder sacar más información de estos, encontrar algo que nos sea de importancia;

- Revisar la extensión de los encabezados
- Comprobar cuántas palabras únicas tiene cada encabezado

	headline	is_sarcastic	article_link
0	former versace store clerk sues over secret 'b...	0	https://www.huffingtonpost.com/entry/versace-b...
1	the 'roseanne' revival catches up to our thorn...	0	https://www.huffingtonpost.com/entry/roseanne-...
2	mom starting to fear son's web series closest ...	1	https://local.theonion.com/mom-starting-to-fea...
3	boehner just wants wife to listen, not come up...	1	https://politics.theonion.com/boehner-just-wan...
4	j.k. rowling wishes snape happy birthday in th...	0	https://www.huffingtonpost.com/entry/jk-rowlin...

Figura 1: Tabla de contenido con los atributos de cada registro en el archivo .json

- Checar si el encabezado tiene algún dígito

```

1 df['headline_count'] = df.headline.apply(lambda x: len(list(x.split
2   ())))
3 df['headline_unique_word_count'] = df.headline.apply(lambda x: len(
4   set(x.split())))
5 df['headline_has_digits'] = df.headline.apply(lambda x: bool(re.
6   search(r'\d', x)))
7 df

```

	headline	is_sarcastic	article_link	headline_count	headline_unique_word_count	headline_has_digits
0	former versace store clerk sues over secret 'b...	0	https://www.huffingtonpost.com/entry/versace-b...	12	12	False
1	the 'roseanne' revival catches up to our thorn...	0	https://www.huffingtonpost.com/entry/roseanne-...	14	14	False
2	mom starting to fear son's web series closest ...	1	https://local.theonion.com/mom-starting-to-fea...	14	13	False
3	boehner just wants wife to listen, not come up...	1	https://politics.theonion.com/boehner-just-wan...	13	13	False
4	j.k. rowling wishes snape happy birthday in th...	0	https://www.huffingtonpost.com/entry/jk-rowlin...	11	11	False
...	...	...	...	...	...	...
26704	american politics in moral free-fall	0	https://www.huffingtonpost.com/entry/american-...	5	5	False
26705	america's best 20 hikes	0	https://www.huffingtonpost.com/entry/americas-...	4	4	True
26706	reparations and obama	0	https://www.huffingtonpost.com/entry/reparatio...	3	3	False
26707	israeli ban targeting boycott supporters raise...	0	https://www.huffingtonpost.com/entry/israeli-b...	8	8	False
26708	gourmet gifts for the foodie 2014	0	https://www.huffingtonpost.com/entry/gourmet-g...	6	6	True

26709 rows x 6 columns

Figura 2: Análisis de los registros

Podemos observar que en total tenemos 26 709 registros de tweets. Ahora, para poder trabajar con estos con estos registros debemos de limpiarlos. Es decir, debemos de eliminar ciertos caracteres que pueden meter ruido en el entrenamiento;

- Signos de puntuación, porque no sabe lidiar con esos `\[[^]]*\]`
- Caracteres especiales que no sean parte del idioma a usar, en este caso es inglés.
- Lematización que es cambiar palabras de tercera persona a primera persona, cambiar palabras en pasado y futuro a presente, poner todo en raíz.

- Stop words que son artículos, preposiciones, pronombres.

```

1 #Removing punctuation marks
2 def remove_punctuations(text):
3     return re.sub('\[[^\]]*\]', '', text)
4
5 #Removing special characters
6 def remove_specialchars(text):
7     return re.sub("[^a-zA-Z0-9]", " ",text)
8
9 #Removal of stopwords and lemmatization
10 def remove_stopwords_and_lemmatization(text):
11     final_text = []
12     text = text.lower() #convert all words into lowercase
13     text = nltk.word_tokenize(text)
14
15     for word in text:
16         if word not in set(stopwords.words('english')):
17             lemma = nltk.WordNetLemmatizer()
18             word = lemma.lemmatize(word)
19             final_text.append(word)
20     return " ".join(final_text)
21
22
23 #Total function
24 def cleaning(text):
25     text = remove_punctuations(text)
26     text = remove_specialchars(text)
27     text = remove_stopwords_and_lemmatization(text)
28     return text
29
30 df['headline']=df['headline'].apply(cleaning)
31
32 df.head()

```

	headline	is_sarcastic	article_link
0	former versace store clerk sue secret black co...	0	https://www.huffingtonpost.com/entry/versace-b...
1	roseanne revival catch thorny political mood b...	0	https://www.huffingtonpost.com/entry/roseanne-...
2	mom starting fear son web series closest thing...	1	https://local.theonion.com/mom-starting-to-fea...
3	boehner want wife listen come alternative debt...	1	https://politics.theonion.com/boehner-just-wan...
4	j k rowling wish snape happy birthday magical way	0	https://www.huffingtonpost.com/entry/jk-rowlin...

Figura 3: Tabla de contenido con los atributos pero ya limpiados.

```

1 df['headline_count'] = df.headline.apply(lambda x: len(list(x.split())))
2 df['headline_unique_word_count'] = df.headline.apply(lambda x: len(set(x.split())))
3 df['headline_has_digits'] = df.headline.apply(lambda x: bool(re.search(r'\d', x)))
4 df

```

	headline	is_sarcastic	article_link	headline_count	headline_unique_word_count	headline_has_digits
0	former versace store clerk sue secret black co...	0	https://www.huffingtonpost.com/entry/versace-b...	10	10	False
1	roseanne revival catch thorny political mood b...	0	https://www.huffingtonpost.com/entry/roseanne-...	8	8	False
2	mom starting fear son web series closest thing...	1	https://local.theonion.com/mom-starting-to-fea...	9	9	False
3	boehner want wife listen come alternative debt...	1	https://politics.theonion.com/boehner-just-wan...	9	9	False
4	j k rowling wish snape happy birthday magical way	0	https://www.huffingtonpost.com/entry/jk-rowlin...	9	9	False
...	...	...	...	...	...	...
26704	american politics moral free fall	0	https://www.huffingtonpost.com/entry/american-...	5	5	False
26705	america best 20 hike	0	https://www.huffingtonpost.com/entry/americas-...	4	4	True
26706	reparation obama	0	https://www.huffingtonpost.com/entry/reparatio...	2	2	False
26707	israeli ban targeting boycott supporter raise ...	0	https://www.huffingtonpost.com/entry/israeli-b...	8	8	False
26708	gourmet gift foodie 2014	0	https://www.huffingtonpost.com/entry/gourmet-g...	4	4	True

26709 rows × 6 columns

Figura 4: Análisis de los registros ya limpiados y acortados.

Ya tenemos los registros listos para poder separarlos en nuestros sets de entrenamiento y prueba. Podemos ver que hubo registros que redujeron su tamaño considerablemente con la eliminación de palabras repetidas y la limpieza.

Podemos seguir haciendo análisis previos a los datos. Esta práctica es una buena manera de poder encontrar algún tipo de patrón específico en los datos y no solo meter datos directamente a un modelo para simplemente tener un resultado.

- ¿Cuáles son las frecuencias de los titulares sarcásticos en comparación con los titulares no sarcásticos?
- ¿Cuál es la longitud de las palabras de los titulares? ¿Para los titulares sarcásticos y no sarcásticos?
- ¿Los titulares sarcásticos utilizan estadísticas (dígitos/números) en su redacción? ¿En comparación con los titulares no sarcásticos?

```

1 sarcastic_dat = df.groupby('is_sarcastic').count()
2 sarcastic_dat.index = ['Non-sarcastic', 'Sarcastic']
3 plt.xlabel('Type of headlines (Sarcastic & Non-sarcastic)')
4 plt.ylabel('Frequencies of headlines')
5 plt.xticks(fontsize=10)
6 plt.title('Frequencies of Sarcastic vs Non-sarcastic headlines')
7 bar_graph = plt.bar(sarcastic_dat.index, sarcastic_dat.
8                       headline_count)
9 bar_graph[1].set_color('r')
10 plt.show()
11 round(sarcastic_dat.headline_count / sarcastic_dat.headline_count.
12        sum(), 2)

```

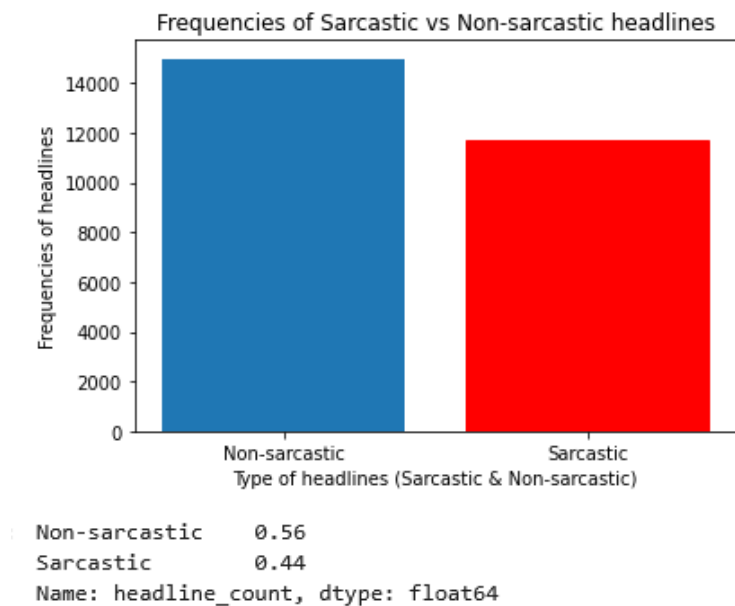


Figura 5: Frecuencia de encabezados sarcásticos y no sarcásticos

```

1 all_dat = df.groupby('headline_count').count()
2 sarcastic_dat1 = df[df.is_sarcastic==1]
3 sarcastic_dat = sarcastic_dat1.groupby('headline_count').count()
4 not_sarcastic_dat1 = df[df.is_sarcastic==0]
5 not_sarcastic_dat = not_sarcastic_dat1.groupby('headline_count').
  count()
6
7 plt.xlabel('Different lengths of headline')
8 plt.ylabel('Frequencies of headline length')
9 plt.xticks(fontsize=10)
10 plt.title('Distribution of headline length for entire dataset')
11 bar_graph = plt.bar(all_dat.index, all_dat.headline)
12 bar_graph[8].set_color('r')
13 plt.axvline(df.headline_count.mean(), color='k', linestyle='dashed',
  , linewidth=1) # median is 10 words in a headline
14 plt.show()

```

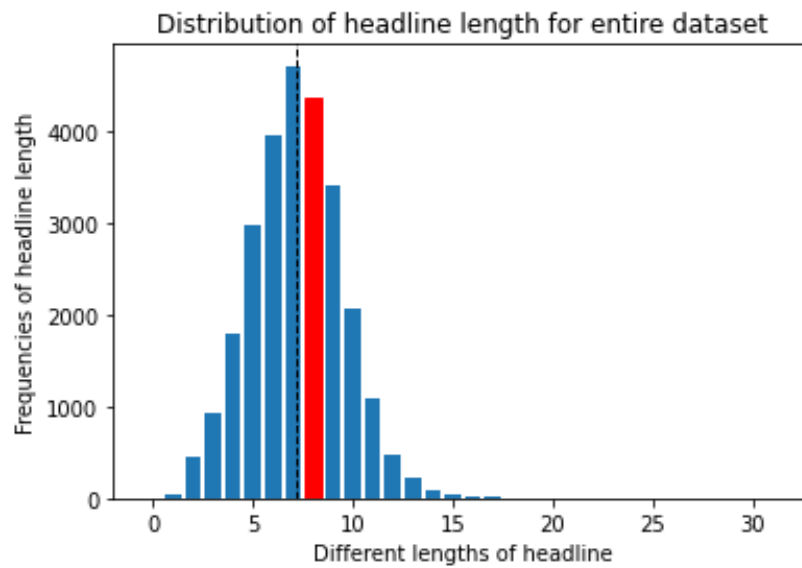



Figura 6: Longitud de los encabezados en todo el dataset

```

1 plt.xlabel('Different lengths of sarcastic headline')
2 plt.ylabel('Frequencies of sarcastic headline length')
3 plt.xticks(fontsize=10)
4 plt.title('Distribution of headline length for sarcastic dataset')
5 bar_graph = plt.bar(sarcastic_dat.index, sarcastic_dat.headline)
6 bar_graph[7].set_color('r')
7 plt.axvline(sarcastic_dat1.headline_count.mean(), color='k',
8             linestyle='dashed', linewidth=1) # median is 10 words in a
           headline
9 plt.show()

```

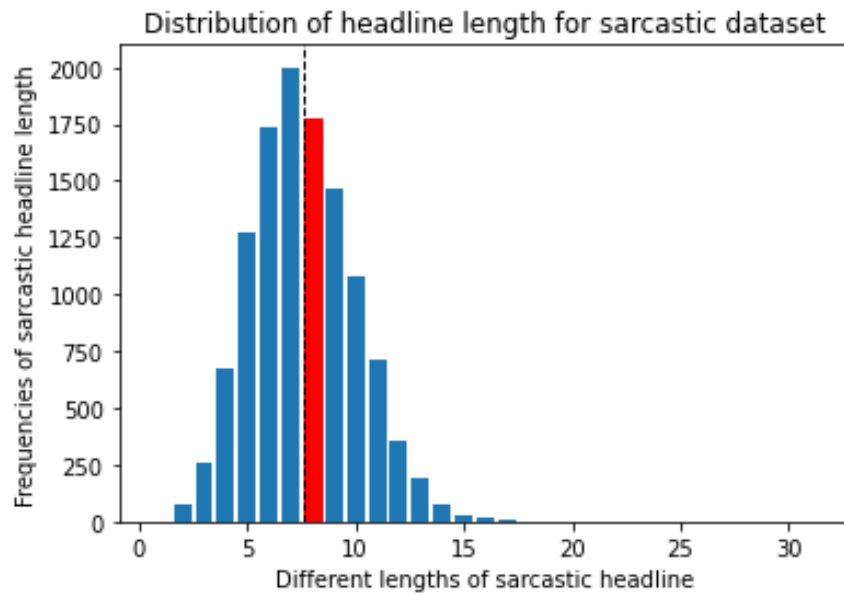


Figura 7: Longitud de los encabezados sarcásticos en todo el dataset

```

1 plt.xlabel('Different lengths of non-sarcastic headline')
2 plt.ylabel('Frequencies of non-sarcastic headline length')
3 plt.xticks(fontsize=10)
4 plt.title('Distribution of headline length for non-sarcastic
5           dataset')
6 bar_graph = plt.bar(not_sarcastic_dat.index, not_sarcastic_dat.
7                     headline)
8 bar_graph[8].set_color('r')
9 plt.axvline(not_sarcastic_dat1.headline_count.mean(), color='k',
10             linestyle='dashed', linewidth=1) # median is 10 words in a
11             headline
12 plt.show()

```

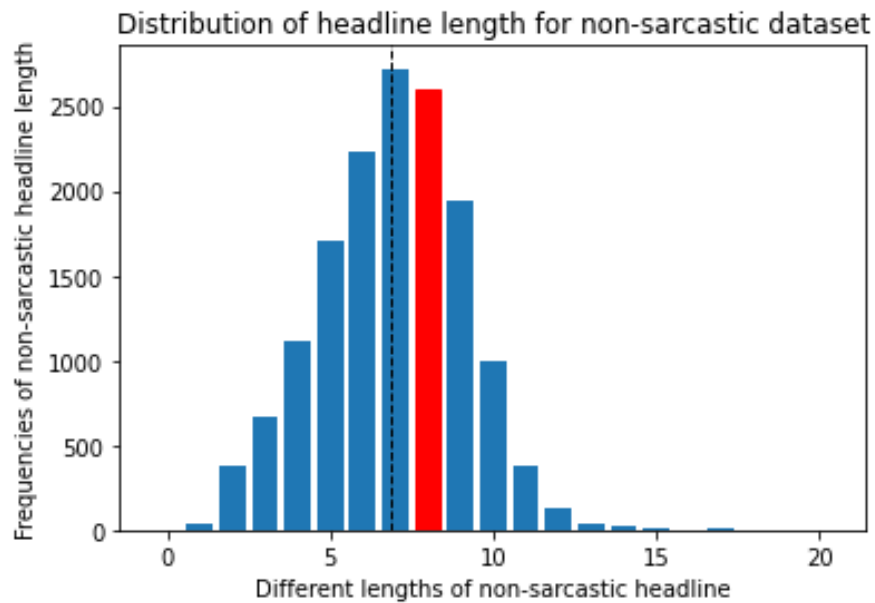


Figura 8: Longitud de los encabezados no sarcásticos en todo el dataset

```

1 digits_dat = df.groupby('headline_has_digits').count()
2 digits_dat.index = ['Has Numbers in Headline', 'Does not have
   Numbers in Headline']
3
4
5 plt.xlabel('Type of headlines')
6 plt.ylabel('Frequencies of headlines')
7 plt.xticks(fontsize=10)
8 plt.title('Frequencies of headlines with Numbers vs No numbers')
9 bar_graph = plt.bar(digits_dat.index, digits_dat.headline /
   digits_dat.headline_count.sum())
10 bar_graph[1].set_color('r')
11 plt.gca().yaxis.set_major_formatter(PercentFormatter(1))
12 plt.show()

```

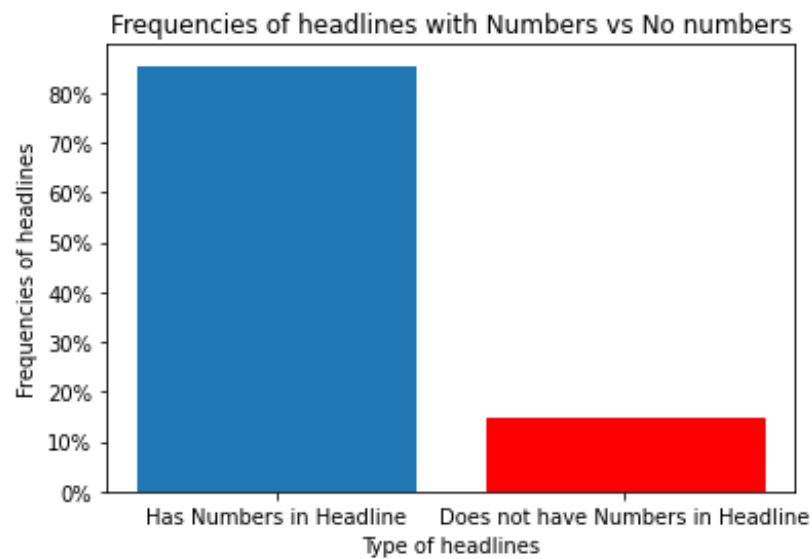


Figura 9: Frecuencia de encabezados con y sin números en todo el dataset

```
1 sarcastic_digits_dat = df[df.is_sarcastic==1].groupby('
    headline_has_digits').count()
2 sarcastic_digits_dat.index = ['Has Numbers in Headline', 'Does not
    have Numbers in Headline']
3
4
5 plt.xlabel('Type of headlines')
6 plt.ylabel('Frequencies of headlines')
7 plt.xticks(fontsize=10)
8 plt.title('Frequencies of Sarcastic headlines with Numbers vs No
    numbers')
9 bar_graph = plt.bar(sarcastic_digits_dat.index,
    sarcastic_digits_dat.headline / sarcastic_digits_dat.
    headline_count.sum())
10 bar_graph[1].set_color('r')
11 plt.gca().yaxis.set_major_formatter(PercentFormatter(1))
12 plt.show()
```

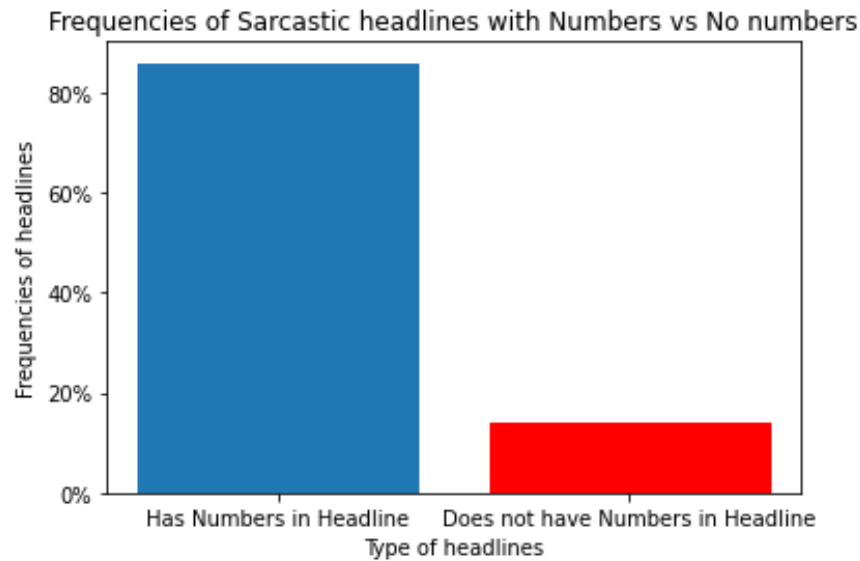


Figura 10: Frecuencia de encabezados sarcásticos con y sin números en todo el dataset

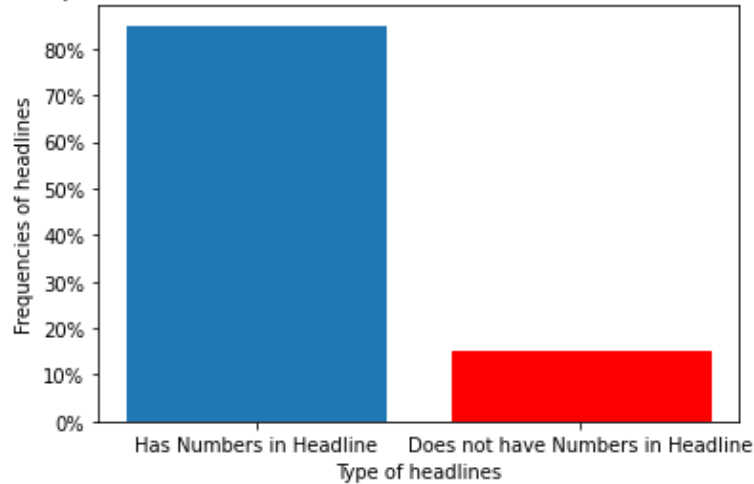
```

1 not_sarcastic_digits_dat = df[df.is_sarcastic==0].groupby('
    headline_has_digits').count()
2 not_sarcastic_digits_dat.index = ['Has Numbers in Headline', 'Does
    not have Numbers in Headline']
3
4
5 plt.xlabel('Type of headlines')
6 plt.ylabel('Frequencies of headlines')
7 plt.xticks(fontsize=10)
8 plt.title('Frequencies of Non-sarcastic headlines with Numbers vs
    No numbers')
9 bar_graph = plt.bar(not_sarcastic_digits_dat.index,
    not_sarcastic_digits_dat.headline / not_sarcastic_digits_dat.
    headline_count.sum())
10 bar_graph[1].set_color('r')
11 plt.gca().yaxis.set_major_formatter(PercentFormatter(1))
12 plt.show()
13
14 print(round(digits_dat.headline / digits_dat.headline_count.sum()
    ,2))
15 print(round(sarcastic_digits_dat.headline / sarcastic_digits_dat.
    headline_count.sum(),2))
16 print(round(not_sarcastic_digits_dat.headline /
    not_sarcastic_digits_dat.headline_count.sum(),2))

1 text = " ".join(review for review in df.headline)
2 print ("There are {} words in the combination of all review.".
    format(len(text)))

```

Frequencies of Non-sarcastic headlines with Numbers vs No numbers



```

Has Numbers in Headline      0.85
Does not have Numbers in Headline  0.15
Name: headline, dtype: float64
Has Numbers in Headline      0.86
Does not have Numbers in Headline  0.14
Name: headline, dtype: float64
Has Numbers in Headline      0.85
Does not have Numbers in Headline  0.15
Name: headline, dtype: float64

```

Figura 11: Frecuencia de encabezados no sarcásticos con y sin números en todo el dataset

```

3
4 There are 1304276 words in the combination of all review.

```

Terminado nuestro análisis podemos proceder a la creación de nuestros sets de entrenamiento y validación.

```

1 train_data, test_data = train_test_split(df[['headline', '
    is_sarcastic']], test_size=0.1) # randomly splitting 10% of
    dataset to be training dataset
2
3 training_sentences = list(train_data['headline'])
4 training_labels = list(train_data['is_sarcastic'])
5
6 testing_sentences = list(test_data['headline'])
7 testing_labels = list(test_data['is_sarcastic'])
8 training_labels_final = np.array(training_labels)
9 testing_labels_final = np.array(testing_labels)

```

Aunque en la primera sección ya se han importado librerías generales a continuación se importan módulos más especializados de TensorFlow relacionados con la construcción del modelo específico para esta actividad.

- Modelos y capas: Sequential, Dense, GRU, Embedding.
- Preprocesamiento de texto: Tokenizer, pad_sequences.
- Optimizadores: Adam, SGD, RMSprop.
- Callbacks: EarlyStopping.

Estas líneas idealmente deberían ubicarse al inicio del proyecto por razones de organización, sin embargo, aquí se presentan en el contexto donde comienzan a utilizarse en el desarrollo del modelo.

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from keras.callbacks import EarlyStopping
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.optimizers import Adam, SGD, RMSprop
6 from tensorflow.keras.layers import Dense, GRU, Embedding
7 from tensorflow.keras.preprocessing.text import Tokenizer
8 from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Esta parte del código se encarga de preparar los datos de texto para el entrenamiento de un modelo al convertir los textos en secuencias numéricas y asegurarse de que todas las secuencias tengan una longitud uniforme. Esto es esencial para que el modelo pueda manejar las entradas de texto de manera efectiva.

- El vocab_size: Define el tamaño del vocabulario, limitando el modelo a las 10,000 palabras más frecuentes en los datos.
- El embedding_dim: Especifica la dimensión del espacio de embedding. En otras palabras, es el tamaño de los vectores que se utilizarán para representar cada palabra en el vocabulario. Cada palabra se representará como un vector de 16 valores numéricos continuos.
- El max_length: Define la longitud máxima de las secuencias de texto. Si un texto es más largo, se truncará; si es más corto, se rellenará.
- El trunc_type='post': Indica que si una secuencia de texto es más larga que max_length, se truncará desde el final
- El tokenizer es una herramienta utilizada en el procesamiento de texto para convertir secuencias de palabras en secuencias de enteros.
- El fit_on_texts ajusta el tokenizador a las oraciones de entrenamiento para construir un índice de palabras.
- El word_index contiene un diccionario que asigna un índice único a cada palabra en el vocabulario.

- El `texts_to_sequences` convierte las oraciones en secuencias de enteros basadas en el índice de palabras.
- El `pad_sequences` asegura que todas las secuencias tengan la misma longitud (`max_length`). Las secuencias más cortas se rellenan con ceros y las más largas se truncan según `trunc_type`.

```

1 vocab_size = 10000
2 embedding_dim = 16
3 max_length = 120
4 trunc_type='post'
5
6 tokenizer = Tokenizer(num_words = vocab_size)
7 tokenizer.fit_on_texts(training_sentences)
8
9 word_index = tokenizer.word_index
10 sequences = tokenizer.texts_to_sequences(training_sentences)
11 padded = pad_sequences(sequences, maxlen=max_length, truncating=
    trunc_type)
12
13 testing_sequences = tokenizer.texts_to_sequences(testing_sentences)
14 testing_padded = pad_sequences(testing_sequences, maxlen=max_length)

```

Ejemplo:

```

1 tokenizer = Tokenizer(num_words=10000)
2
3 tokenizer.fit_on_texts(["I love machine learning", "Machine
    learning is fun"])
4
5 word_index = tokenizer.word_index
6
7 {'machine': 1, 'learning': 2, 'i': 3, 'love': 4, 'is': 5, 'fun': 6}
8
9 sequences = tokenizer.texts_to_sequences(["I love machine learning"
    ]) [[3, 4, 1, 2]]
10
11 padded = pad_sequences(sequences, maxlen, truncating) [[0, 0, 0, 0,
    3, 4, 1, 2, 0, 0]]

```


4.4. Construir el Modelo.

4.4.1. RNN + GRU + Regularización Lasso (L1)

GRU es un tipo de neurona especial que va a guardar información. Va a permitir las conexiones recurrentes. Utilizan 2 tipos de neuronas; gru y lstm.

Entre más capas tiene una red, se hace mayor la probabilidad de unos problemas; el gradiente se hace 0 o se va al infinito.

Son especiales ya que almacenan información de los gradientes antes de la derivada. Lo van conservando (pasando) a las siguiente unidades o lo va desgastando de una forma menor.

- La primera capa convierte los índices de las palabras en vectores densos de tamaño `embedding_dim`
- La capa GRU (Gated Recurrent Unit) es una celda recurrente que se utiliza para manejar secuencias de datos y mantener dependencias a largo plazo. Procesa datos secuenciales y mantiene una memoria interna que se actualiza con cada paso de tiempo. Guarda los pesos que mejor le han servido, lo evalúa y decide si ese peso debe conservarse o reiniciarlo para calcularlo de nuevo y mejorarlo.
- La capa GRU está envuelta en una capa Bidireccional, lo que significa que el modelo procesa la secuencia en ambas direcciones (de adelante hacia atrás y de atrás hacia adelante), mejorando así la comprensión del contexto en ambas direcciones.
- La capa densa con 100 unidades y activación ReLU, con regularización L1 aplicada a los pesos. Esta normalización, a la hora de calcular los pesos de la red, agrega una penalización basada en la suma de los valores absolutos de los coeficientes de los parámetros del modelo. Fomenta la simplicidad y la esparsidad en el modelo, ayudando a prevenir el sobreajuste y a realizar una selección de características implícita. Elimina atributos de entrada irrelevantes.
- La siguiente capa normaliza las salidas de la capa anterior para mejorar la estabilidad del entrenamiento y acelerar el proceso de aprendizaje.
- La capa Dropout aplica una tasa de abandono del 50 % durante el entrenamiento para evitar el sobreajuste al .^apagar.^aleatoriamente un porcentaje de las neuronas durante cada época de entrenamiento.
- La capa de salida tiene una sola unidad con activación sigmoide, lo que produce una probabilidad entre 0 y 1 para la clasificación binaria (sarcasmo vs. no sarcasmo).

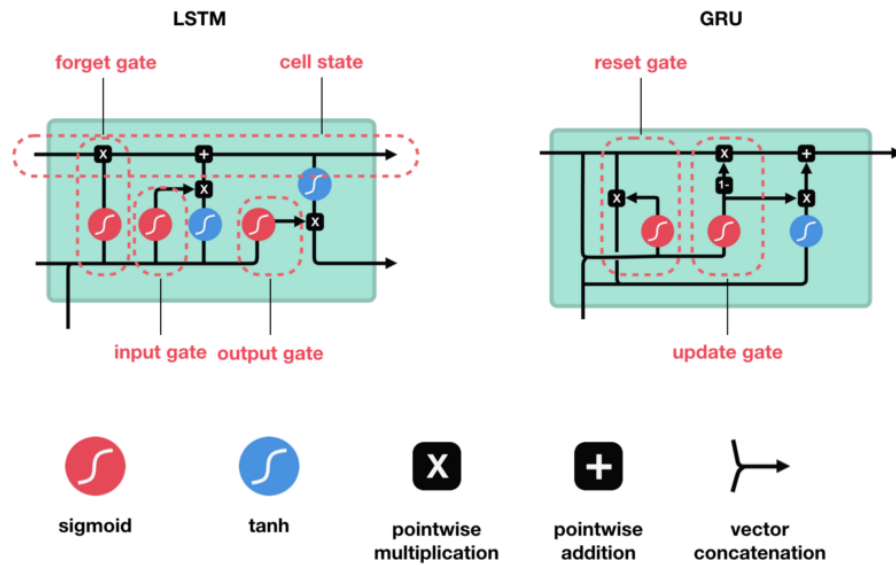


Figura 12: Gráfico que muestra la función de la LSTM y GRU

```

1 model_gru_l1 = keras.Sequential([
2     keras.layers.Embedding(vocab_size, embedding_dim, input_length=
3     max_length),
4     keras.layers.Bidirectional(keras.layers.GRU(32)),
5     keras.layers.Dense(50, kernel_regularizer=keras.regularizers.l1
6     (0.007), activation='relu'),
7     #keras.layers.BatchNormalization(),
8     keras.layers.Dropout(0.5),
9     keras.layers.Dense(1, kernel_regularizer=keras.regularizers.l1
10    (0.007), activation='sigmoid')
11 ])
12 model_gru_l1.compile(loss='binary_crossentropy', optimizer='adam',
13     metrics=['accuracy'])
14 model_gru_l1.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 120, 16)	160000
bidirectional (Bidirectional)	(None, 64)	9600
dense (Dense)	(None, 50)	3250
dropout (Dropout)	(None, 50)	0
dense_1 (Dense)	(None, 1)	51

=====
Total params: 172,901
Trainable params: 172,901
Non-trainable params: 0
=====

Figura 13: Estructura del modelo

```
1 num_epochs = 5
2
3 early_stopping = EarlyStopping(monitor='val_loss', patience=2)
4
5 history_gru_l1 = model_gru_l1.fit(
6     padded,
7     training_labels_final,
8     epochs=num_epochs,
9     batch_size=256,
10    validation_data=(testing_padded,
11        testing_labels_final),
12    callbacks=[early_stopping]
13 )
```

Epoch 1/5
94/94 [=====] - 9s 30ms/step - loss: 2.3518 - accuracy: 0.5681 - val_loss: 1.5305 - val_accuracy: 0.6342
Epoch 2/5
94/94 [=====] - 2s 19ms/step - loss: 0.9958 - accuracy: 0.7843 - val_loss: 0.6634 - val_accuracy: 0.8008
Epoch 3/5
94/94 [=====] - 2s 18ms/step - loss: 0.5361 - accuracy: 0.8603 - val_loss: 0.5733 - val_accuracy: 0.8016
Epoch 4/5
94/94 [=====] - 2s 18ms/step - loss: 0.4558 - accuracy: 0.8924 - val_loss: 0.5790 - val_accuracy: 0.7967
Epoch 5/5
94/94 [=====] - 2s 18ms/step - loss: 0.4128 - accuracy: 0.9143 - val_loss: 0.6017 - val_accuracy: 0.7971

Figura 14: Entrenamiento del modelo

```

1 def plot_graphs(history, string):
2     plt.plot(history.history[string])
3     plt.plot(history.history['val_'+string])
4     plt.xlabel("Epochs")
5     plt.ylabel(string)
6     plt.legend([string, 'val_'+string])
7     plt.show()
8
9 plot_graphs(history_gru_l1, 'accuracy')
10 plot_graphs(history_gru_l1, 'loss')
11 plt.show()

```

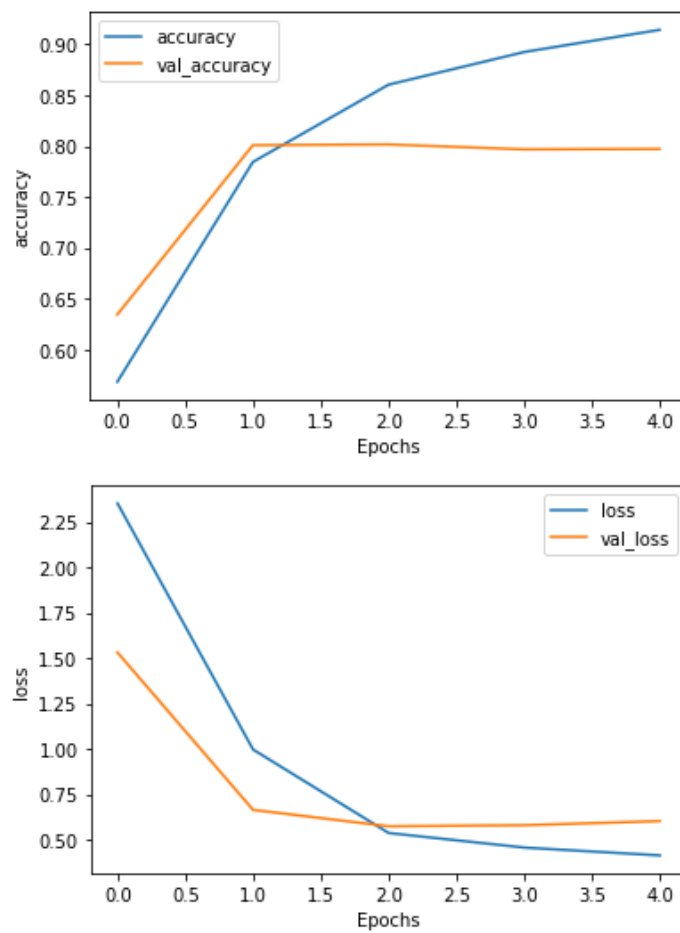


Figura 15: Resultado del entrenamiento del modelo

Probar RNN + GRU + Regularización Lasso (L1)

```

1 # Cargar el archivo JSON
2 df_gru_l1 = pd.read_json(Path + "Sarcasm_Headlines_Dataset/
   Sarcasm_Headlines_Dataset_v2.json", lines=True)
3
4 # Aplicar limpieza a los titulares
5 df_gru_l1['headline'] = df_gru_l1['headline'].apply(cleaning)
6
7 # Convertir los titulares limpiados en secuencias y aplicar padding
8 sequences_gru_l1 = tokenizer.texts_to_sequences(df_gru_l1['headline
   '])
9 padded_gru_l1 = pad_sequences(sequences_gru_l1, maxlen=max_length,
   truncating=trunc_type)

1 predictions_gru_l1 = model_gru_l1.predict(padded_gru_l1)

1 # Convertir las predicciones en etiquetas binarizadas (0 o 1)
2 predicted_labels = (predictions_gru_l1 > 0.5).astype(int)
3
4 # Agregar las predicciones al DataFrame original para visualizacion
5 df_gru_l1['predicted_is_sarcastic'] = predicted_labels
6
7 # Mostrar algunas filas con los titulares y las predicciones
8 print(df_gru_l1[['headline', 'predicted_is_sarcastic', '
   is_sarcastic']].head())

```

	headline	predicted_is_sarcastic	\
0	thirtysomething scientist unveil doomsday cloc...	1	
1	dem rep totally nail congress falling short ge...	0	
2	eat veggie 9 deliciously different recipe	0	
3	inclement weather prevents liar getting work	1	
4	mother come pretty close using word streaming ...	1	

	is_sarcastic
0	1
1	0
2	0
3	1
4	1

Figura 16: Resultado de predicción del primero modelo

4.4.2. RNN + GRU + Regularización Ridge (L2)

A diferencia de la regularización L1, la L2 agrega una penalización a la función de pérdida que se basa en la suma de los cuadrados de los coeficientes del modelo. Reduce el tamaño de todos los coeficientes, evitando que se vuelvan demasiado grandes, lo que ayuda a suavizar el modelo y a prevenir el sobreajuste.

```
1 model_gru_l2 = keras.Sequential([
2     keras.layers.Embedding(vocab_size, embedding_dim, input_length=
3     max_length),
4     keras.layers.Bidirectional(keras.layers.GRU(32)),
5     keras.layers.Dense(100, kernel_regularizer=keras.regularizers.
6     l2(0.007), activation='relu'),
7     #keras.layers.BatchNormalization(),
8     keras.layers.Dropout(0.5),
9     keras.layers.Dense(1, kernel_regularizer=keras.regularizers.l2
10    (0.007), activation='sigmoid')
11 ])
12 model_gru_l2.compile(loss='binary_crossentropy', optimizer='adam',
13     metrics=['accuracy'])
14 model_gru_l2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 120, 16)	160000
bidirectional_1 (Bidirectional)	(None, 64)	9600
dense_2 (Dense)	(None, 100)	6500
dropout_1 (Dropout)	(None, 100)	0
dense_3 (Dense)	(None, 1)	101
Total params: 176,201		
Trainable params: 176,201		
Non-trainable params: 0		

Figura 17: Estructura del modelo

```

1 num_epochs = 5
2 history_gru_l2 = model_gru_l2.fit(
3     padded,
4     training_labels_final,
5     epochs=num_epochs,
6     batch_size=256,
7     validation_data=(testing_padded,
8         testing_labels_final)
9 )

```

```

Epoch 1/5
94/94 [=====] - 5s 29ms/step - loss: 0.9586 - accuracy: 0.5684 - val_loss: 0.7339 - val_accuracy: 0.6683
Epoch 2/5
94/94 [=====] - 2s 20ms/step - loss: 0.5299 - accuracy: 0.7951 - val_loss: 0.4989 - val_accuracy: 0.7956
Epoch 3/5
94/94 [=====] - 2s 18ms/step - loss: 0.3601 - accuracy: 0.8699 - val_loss: 0.4677 - val_accuracy: 0.8046
Epoch 4/5
94/94 [=====] - 2s 18ms/step - loss: 0.2967 - accuracy: 0.8979 - val_loss: 0.4929 - val_accuracy: 0.8004
Epoch 5/5
94/94 [=====] - 2s 18ms/step - loss: 0.2569 - accuracy: 0.9175 - val_loss: 0.5333 - val_accuracy: 0.8001

```

Figura 18: Entrenamiento del modelo

```

1 plot_graphs(history_gru_l2, 'accuracy')
2 plot_graphs(history_gru_l2, 'loss')
3 plt.show()

```

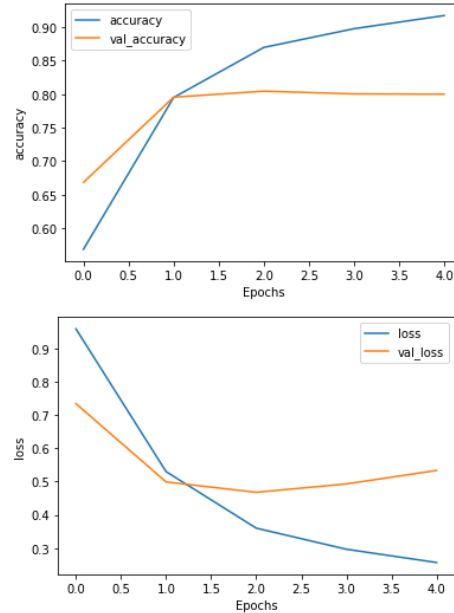


Figura 19: Resultado del entrenamiento del modelo

Probar RNN + GRU + Regularización Ridge (L2)

```

1 # Cargar el archivo JSON
2 df_gru_l2 = pd.read_json(Path + "Sarcasm_Headlines_Dataset/
   Sarcasm_Headlines_Dataset_v2.json", lines=True)
3
4 # Aplicar limpieza a los titulares
5 df_gru_l2['headline'] = df_gru_l2['headline'].apply(cleaning)
6
7 # Convertir los titulares limpiados en secuencias y aplicar padding
8 sequences_gru_l2 = tokenizer.texts_to_sequences(df_gru_l2['headline
   '])
9 padded_gru_l2 = pad_sequences(sequences_gru_l2, maxlen=max_length,
   truncating=trunc_type)

1 predictions_gru_l2 = model_gru_l2.predict(padded_gru_l2)

2 # Convertir las predicciones en etiquetas binarizadas (0 o 1)
3 predicted_labels = (predictions_gru_l2 > 0.5).astype(int)
4
5 # Agregar las predicciones al DataFrame original para visualización
6 df_gru_l2['predicted_is_sarcastic'] = predicted_labels
7
8 # Mostrar algunas filas con los titulares y las predicciones
9 print(df_gru_l2[['headline', 'predicted_is_sarcastic', '
   is_sarcastic']].head())

```

	headline	predicted_is_sarcastic	\
0	thirtysomething scientist unveil doomsday cloc...	1	
1	dem rep totally nail congress falling short ge...	0	
2	eat veggie 9 deliciously different recipe	0	
3	inclement weather prevents liar getting work	1	
4	mother come pretty close using word streaming ...	1	

	is_sarcastic
0	1
1	0
2	0
3	1
4	1

Figura 20: Resultado de predicción del segundo modelo

4.4.3. RNN + LSTM + Regularización Lasso (L1)

Ahora en vez de usar una capa GRU, usaremos una capa de Long Short-Term Memory (LSTM): un peso en la memoria de corto plazo que estará actualizando en cada iteración. Un peso a largo plazo que se estará guardando en cada iteración, es el mejor peso de la memoria a corto plazo que ha servido, y un peso que debe ser olvidado ya que no ayudó al entrenamiento.

```
1 model_lstm_l1 = keras.Sequential([
2     keras.layers.Embedding(vocab_size, embedding_dim, input_length=
3     max_length),
4     keras.layers.Bidirectional(keras.layers.LSTM(32)),
5     keras.layers.Dense(100, kernel_regularizer=keras.regularizers.
6     l1(0.007), activation='relu'),
7     #keras.layers.BatchNormalization(),
8     keras.layers.Dropout(0.5),
9     keras.layers.Dense(1, kernel_regularizer=keras.regularizers.l1
10    (0.007), activation='sigmoid')
11 ])
12 model_lstm_l1.compile(loss='binary_crossentropy', optimizer='adam',
13    metrics=['accuracy'])
14 model_lstm_l1.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 120, 16)	160000
bidirectional_2 (Bidirectional)	(None, 64)	12544
dense_4 (Dense)	(None, 100)	6500
dropout_2 (Dropout)	(None, 100)	0
dense_5 (Dense)	(None, 1)	101
Total params: 179,145		
Trainable params: 179,145		
Non-trainable params: 0		

Figura 21: Estructura del modelo

```

1 num_epochs = 5
2 history_lstm_l1 = model_lstm_l1.fit(
3     padded,
4     training_labels_final,
5     epochs=num_epochs,
6     batch_size=256,
7     validation_data=(testing_padded,
8         testing_labels_final)
9 )

```

```

Epoch 1/5
94/94 [=====] - 6s 31ms/step - loss: 3.2789 - accuracy: 0.5612 - val_loss: 1.8262 - val_accuracy: 0.5608
Epoch 2/5
94/94 [=====] - 2s 21ms/step - loss: 1.0810 - accuracy: 0.6529 - val_loss: 0.7017 - val_accuracy: 0.6952
Epoch 3/5
94/94 [=====] - 2s 20ms/step - loss: 0.6091 - accuracy: 0.8113 - val_loss: 0.5903 - val_accuracy: 0.7948
Epoch 4/5
94/94 [=====] - 2s 20ms/step - loss: 0.5139 - accuracy: 0.8654 - val_loss: 0.5811 - val_accuracy: 0.7918
Epoch 5/5
94/94 [=====] - 2s 20ms/step - loss: 0.4652 - accuracy: 0.8895 - val_loss: 0.5775 - val_accuracy: 0.7960

```

Figura 22: Entrenamiento del modelo

```

1 plot_graphs(history_lstm_l1, 'accuracy')
2 plot_graphs(history_lstm_l1, 'loss')
3 plt.show()

```

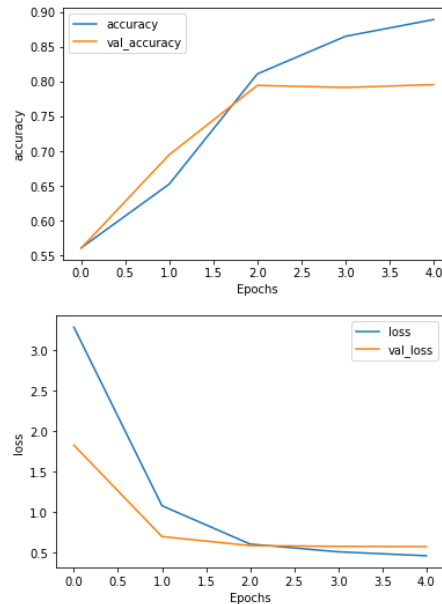


Figura 23: Resultado del entrenamiento del modelo

Probando RNN + LSTM + Regularización Lasso (L1)

```

1 # Cargar el archivo JSON
2 df_lstm_l1 = pd.read_json(Path + "Sarcasm_Headlines_Dataset/
   Sarcasm_Headlines_Dataset_v2.json", lines=True)
3
4 # Aplicar limpieza a los titulares
5 df_lstm_l1['headline'] = df_lstm_l1['headline'].apply(cleaning)
6
7 # Convertir los titulares limpiados en secuencias y aplicar padding
8 sequences_lstm_l1 = tokenizer.texts_to_sequences(df_lstm_l1['
   headline'])
9 padded_lstm_l1 = pad_sequences(sequences_lstm_l1, maxlen=max_length
   , truncating=trunc_type)

1 predictions_lstm_l1 = model_lstm_l1.predict(padded_lstm_l1)

1 # Convertir las predicciones en etiquetas binarizadas (0 o 1)
2 predicted_labels = (predictions_lstm_l1 > 0.5).astype(int)
3
4 # Agregar las predicciones al DataFrame original para visualización
5 df_lstm_l1['predicted_is_sarcastic'] = predicted_labels
6
7 # Mostrar algunas filas con los titulares y las predicciones
8 print(df_lstm_l1[['headline', 'predicted_is_sarcastic', '
   is_sarcastic']].head())

```

	headline	predicted_is_sarcastic	\
0	thirtysomething scientist unveil doomsday cloc...	1	
1	dem rep totally nail congress falling short ge...	0	
2	eat veggie 9 deliciously different recipe	0	
3	inclement weather prevents liar getting work	0	
4	mother come pretty close using word streaming ...	1	

	is_sarcastic
0	1
1	0
2	0
3	1
4	1

Figura 24: Resultado de predicción del tercer modelo

4.4.4. RNN + LSTM + Regularización Ridge (L2)

```
1 model_lstm_l2 = keras.Sequential([
2     keras.layers.Embedding(vocab_size, embedding_dim, input_length=
3     max_length),
4     keras.layers.Bidirectional(keras.layers.LSTM(32)),
5     keras.layers.Dense(100, kernel_regularizer=keras.regularizers.
6     l2(0.007), activation='relu'),
7     #keras.layers.BatchNormalization(),
8     keras.layers.Dropout(0.5),
9     keras.layers.Dense(1, kernel_regularizer=keras.regularizers.l2
10    (0.007), activation='sigmoid')
11 ])
12 model_lstm_l2.compile(loss='binary_crossentropy', optimizer='adam',
13    metrics=['accuracy'])
14 model_lstm_l2.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 120, 16)	160000
bidirectional_3 (Bidirectional)	(None, 64)	12544
dense_6 (Dense)	(None, 100)	6500
dropout_3 (Dropout)	(None, 100)	0
dense_7 (Dense)	(None, 1)	101
Total params: 179,145		
Trainable params: 179,145		
Non-trainable params: 0		

Figura 25: Estructura del modelo

```

1 num_epochs = 5
2 history_lstm_l2 = model_lstm_l2.fit(
3     padded,
4     training_labels_final,
5     epochs=num_epochs,
6     batch_size=256,
7     validation_data=(testing_padded,
8         testing_labels_final)
9 )

```

Epoch 1/5
94/94 [=====] - 6s 35ms/step - loss: 0.9491 - accuracy: 0.5840 - val_loss: 0.7247 - val_accuracy: 0.6496
Epoch 2/5
94/94 [=====] - 2s 23ms/step - loss: 0.5159 - accuracy: 0.8088 - val_loss: 0.4807 - val_accuracy: 0.7978
Epoch 3/5
94/94 [=====] - 2s 21ms/step - loss: 0.3650 - accuracy: 0.8698 - val_loss: 0.4864 - val_accuracy: 0.7926
Epoch 4/5
94/94 [=====] - 2s 21ms/step - loss: 0.3059 - accuracy: 0.8957 - val_loss: 0.5088 - val_accuracy: 0.7941
Epoch 5/5
94/94 [=====] - 2s 23ms/step - loss: 0.2756 - accuracy: 0.9103 - val_loss: 0.5242 - val_accuracy: 0.7937

Figura 26: Entrenamiento del modelo

```

1 plot_graphs(history_lstm_l2, 'accuracy')
2 plot_graphs(history_lstm_l2, 'loss')
3 plt.show()

```

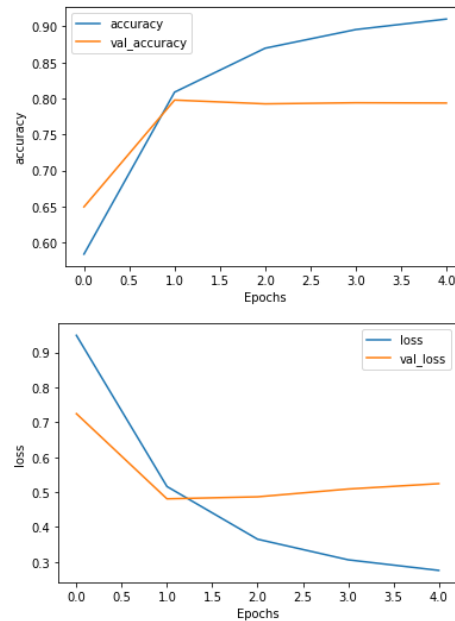


Figura 27: Resultado del entrenamiento del modelo

Probando RNN + LSTM + Regularización Ridge (L2))

```

1 # Cargar el archivo JSON
2 df_lstm_l2 = pd.read_json(Path + "Sarcasm_Headlines_Dataset/
   Sarcasm_Headlines_Dataset_v2.json", lines=True)
3
4 # Aplicar limpieza a los titulares
5 df_lstm_l2['headline'] = df_lstm_l2['headline'].apply(cleaning)
6
7 # Convertir los titulares limpiados en secuencias y aplicar padding
8 sequences_lstm_l2 = tokenizer.texts_to_sequences(df_lstm_l2['
   headline'])
9 padded_lstm_l2 = pad_sequences(sequences_lstm_l2, maxlen=max_length
   , truncating=trunc_type)

1 predictions_lstm_l2 = model_lstm_l2.predict(padded_lstm_l2)

1 # Convertir las predicciones en etiquetas binarizadas (0 o 1)
2 predicted_labels = (predictions_lstm_l2 > 0.5).astype(int)
3
4 # Agregar las predicciones al DataFrame original para visualización
5 df_lstm_l2['predicted_is_sarcastic'] = predicted_labels
6
7 # Mostrar algunas filas con los titulares y las predicciones
8 print(df_lstm_l2[['headline', 'predicted_is_sarcastic', '
   is_sarcastic']].head())

```

	headline	predicted_is_sarcastic	\
0	thirtysomething scientist unveil doomsday cloc...	1	
1	dem rep totally nail congress falling short ge...	0	
2	eat veggie 9 deliciously different recipe	0	
3	inclement weather prevents liar getting work	1	
4	mother come pretty close using word streaming ...	1	

	is_sarcastic
0	1
1	0
2	0
3	1
4	1

Figura 28: Resultado de predicción del cuarto modelo

4.5. Análisis de errores.

Los resultados de exactitud obtenidos son los siguientes:

- RNN + GRU + L1: 0.7971
- RNN + GRU + L2: 0.8001
- RNN + LSTM + L1: 0.7960
- RNN + LSTM + L2: 0.7937

Los modelos con celdas GRU presentan una ligera superioridad respecto a los modelos LSTM, aunque las diferencias no son drásticas. Esto puede atribuirse a que GRU, al tener una estructura más simple que LSTM (menos puertas), puede generalizar mejor en datasets no muy grandes o menos complejos.

En el caso de GRU, la regularización L2 permite alcanzar mejores niveles de exactitud que L1, lo cual sugiere que L2 contribuye a una mejor generalización del modelo en este problema específico. Esto concuerda con literatura donde L2 tiende a suavizar los pesos en lugar de forzarlos a cero (como hace L1), lo que puede beneficiar modelos recurrentes.

Aunque las diferencias entre los modelos son pequeñas (en el orden de 0.003 a 0.006), la consistencia de los resultados sugiere una tendencia clara: GRU + L2 es la combinación más estable y efectiva en este conjunto de experimentos.

4.6. Implementación.

Consulta el Notebook de Jupyter en la carpeta de **Practica 4** llamado *TF-Sarcasmo.ipynb*